CSE225 DATA STRUCTURES Student Name:_____

Borahan Tümer, Ph.D. **MIDTERM ANSWERS** Student Number:_____

1. This exam includes **5** questions.

2. This is an **open-book and open-notes exam**.

3. Time for this exam is **2.5 hours.**

4. You may use a calculator.

5. Do not forget to **write down your name and number on each page**.

6. Please answer the questions in empty spaces left below the questions
   You may use the back of your papers if necessary.

7. You may get partial points. Work neatly and show all your work.

8. You may keep this page if you like. Good luck!!! ☺

Karadut
Karadutum, çatal karam, çingenem
Nar tanem, nur tanem, bir tanem.
Ağaç isem dalımsın salkım saçak
Petek isem balımsın ağulum
Günahımsın, vebalimsin.

Dili mercan, dizi mercan, dişi mercan
Yoluna bir can koyduğum
Gökte ararken yerde olduğum.
Karadutum, çatal karam, çingenem
Daha nem olacaktın bir tanem.
Gülen ayvam, ağlayan narımsın.
Kadınım, kısrağım karımsın.
Bedri Rahmi Eyuboğlu

Saklambaç
Geminin yarısından çoğu
Suyun içinde gizli.
Ağacın kökleri toprağın içinde
Boyundan büyük!!!... Gizli!...
Geceler gündüzlerden uzun
Karanlık... Zifiri... Gizli!...
Sır tutan kişi namuslu
Namuslu kişi..... Gizli!...
Alan gizli... Veren gizli...

Desene saklambaç oynamağa gelmişiz
bu dünyaya!!!
Bedri Rahmi Eyuboğlu

Vatan İçin.....
Neler yapmadık şu vatan için!
Kimimiz öldük;
Kimimiz nutuk söyledik.
Orhan Veli Kanık

Hay Kay.....Gemliğe doğru
Yosun kokusu
Ve bir tabak karides
Sandıkburnu'nda
Orhan Veli Kanık

Denizi göreceksin;
Sakın şaşırma!
Orhan Veli Kanık

Anadolu
Beşikler vermişim Nuh'a,
Salıncaklar, hamaklar;
Havva Anan dünkü çocuk sayılır,
Anadoluyum ben.
Tanıyor musun?
Ahmed Arif

Geçme namert köprüsünden,
Ko aparsın su seni.
Yatma tilki gölgesinde,
Ko yesin aslan seni.
Diyarbakırlı Sait Paşa

Q1. (20pts)
Solve the following recurrences and find $T(n)$!

a. (10pts)

$T(n) = 2\,T(n-1)+n+2^n$; *subject to* $T(0)=0$ *and* $n \geq 1$

$T(n) - 2\,T(n-1) = n+2^n$;

C.E.: $(x-2)(x-1)^2(x-2)=0$

$T(n)=c_1 + c_2 n + c_3 2^n + c_4 n 2^n$

Given $T(0)=0; \Rightarrow T(1)=3;\ T(2)=12;\ T(3)=35$

$$\left.\begin{array}{llll} c_1 + & + c_3 + & = 0 & for \quad n=0 \\ c_1 + c_2 & + 2c_3 + 2c_4 = 3 & for & n=1 \\ c_1 + 2c_2 & + 4c_3 + 8c_4 = 12 & for & n=2 \\ c_1 + 3c_2 & + 8c_3 + 24c_4 = 35 & for & n=3 \end{array}\right\} c_1 = -2; c_2 = -1; c_3 = 2; c_4 = 1$$

Hence,
T(n)= -2 - n+ 2^{n+1} + n2^n

b. (10pts)
T(n)=T(n/3)+nlog_3n; subject to T(1)=0

Change of Variable: $n=3^k$
Hence,
Recurrence: *T(k)=T(k-1)+ 3^kk;*
C.E.: *$(x-1)(x-3)^2 = 0$*
Hence *T(k)= c_1+ $c_2 3^k$ + c_3 k3^k*
Initial Values: *Given for n=1 (or k=0) T(1)=0; ⇒for n=3 or k=1 T(3)=3; for n=9 or k=2 T(9)=21;*

$$\left.\begin{array}{lll} c_1 + c_2 = 0 & for & k=0 \\ c_1 + 3c_2 + 3c_3 = 3 & for & k=1 \\ c_1 + 3c_2 + 3c_3 = 3 & for & k=2 \end{array}\right\} c_1 = \frac{3}{4}; c_2 = -\frac{3}{4}; c_3 = \frac{3}{2};$$

Hence *T(k)=3/4-3/4 3^k + 3/2 k3^k*
Change of Var.: *n=3^k ⇔ k=log_3n*
Hence ***T(n)=3/4 - 3/4n + 3/2n log_3n***

Q2. (20pts)
Consider the following piece of code:

```
.
.
res=0;
for (i=1; i<=n; i++)
   for (j=1; j<=i; j*=2) {
      if (j mod i < 8) res+=2;
   }
printf("res=%d",res)
.
.
```

a. (10pts)
For ***n=16***, how many times is the condition (j mod i < 8) executed?
Show your calculations!
Answer:
The condition is checked once for every turn of the inner for loop. Hence, let us find out the number of tl
execution of the statement block of the inner for loop:

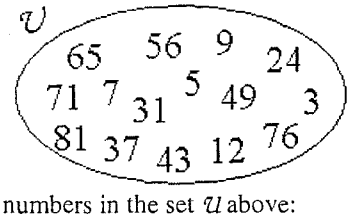
a) $$\sum_{i=1}^{16}\sum_{j=1}^{\lfloor \log_2 i \rfloor+1} 1 = 1+2+2+\underbrace{3+\cdots+3}_{4}+\underbrace{4+\cdots+4}_{8}+5 = 54$$

b. (10pts)
In terms of n, what value of res is printed at the printf statement after
the loop? Show your calculations!
Answer:
For $i=1$ the statement within the if structure is executed once. For $i=2$ and $i=3$, it is executed twice. For e
of the remaining values of i (i.e., $4<i\le n$), the statement within the if structure is executed three times. On
at values $i=2^k$, k an integer, the statement within the if structure is executed one additional time. Hence, t
total number of execution of the statement is

$$S = \begin{cases} 2n-1 & for \quad 1 \le n \le 3 \\ 3(n-2)+2+\left\lfloor \log_2 \frac{n}{4} \right\rfloor = 3n-4+\lfloor \log_2 n \rfloor - 2 & for \quad n \ge 4 \end{cases} \text{ times.}$$

Each execution adds 2 to the current value of res. Starting from res=0,
We get res=2*S printed out.

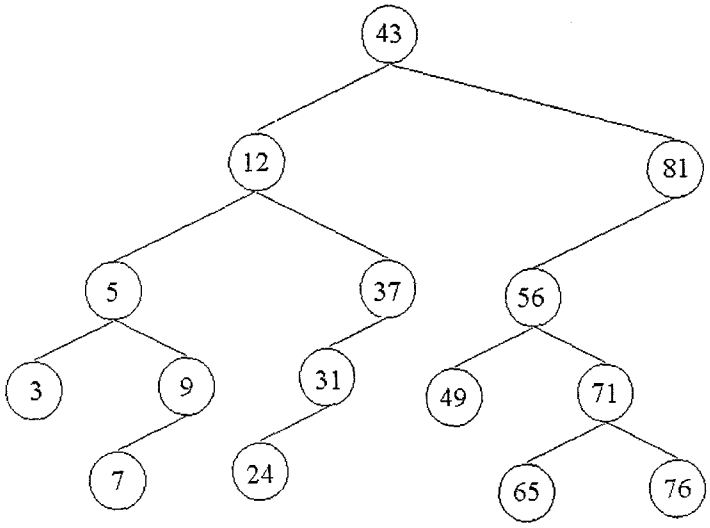
Q3. (40pts)



Consider the numbers in the set $\mathcal{U}$ above:

(10pts) (5+5)
a. From what sequence of the above numbers can the binary search tree (BST) below be constructed? Provi
the sequence and fill in the numbers! Is there any other sequence that provides the same BST? If not, why
not? If yes, give an example!
Two sample sequences providing the following BST:
43, 12, 5, 9, 81, 7, 37, 31, 56, 71, 24, 49, 65, 76, 3

43, 81, 12, 56, 49, 37, 5, 31, 24, 3, 9, 7, 71, 65, 76



(10pts)

b. Assuming that traversing one edge to a child and comparing the searched value with the data in a node is 1 time unit (tu), what are the best-case, average-case and worst-case search times of the BST above in (a) ?

t_{best} = **1tu** (search for the root node)
$t_{average}$ = (1*1+2*2+3*3+4*5+5*4)/12=54/12=**3.6tu**
t_{worst} = **5tu** (search for any node at the deepest level of the tree)

c. (10pts) (5+5)

1. Propose a sequence of the numbers in $\mathcal{U}$ from which a (BST) with best (i.e., shortest) average-c and best (i.e., shortest) worst-case search times can be constructed.

2. Construct the binary search tree (BST) from the sequence you proposed above in (c.1.) and compute the average-case and worst case search times of this specific BST.

c.1. A complete BST with 15 elements is the only tree with the given properties. Hence, any sequence that provides a complete BST is the correct answer to (c.1).

37, 9, 24, 65, 49, 76, 43, 5, 3, 7, 12, 31, 56, 71, 81

c.2. A complete BST is constructed from the above sequence.

t_{best} = **1tu** (search for the root node)
$t_{average}$ = (1*1+2*2+3*4+4*8)/15=49/15=3.**267tu**
t_{worst} = 4**tu** (search for any node at the deepest level of the tree)

(10pts)

d. . How many different BSTs may you construct from the above set of numbers with the same average-case an worst-case search times as those of the BST given in (a)? Show your calculations!

This question is meant to ask all BSTs that have the same number of nodes in each depth level as the BST given in (a) and have the same $t_{average}$ and t_{worst}.

The answer to this question is the product of the combinations of the placement of existing nodes in availal child pointers at each depth level.
That is, we have only one available space at level 0 (root level). Hence, there is only one way we can put o node in this level. In level one, there is the left and right child pointers (i.e., 2 available positions) to put tw nodes. That is, we can again place two nodes in two positions in one way. In level 2, we have three nodes place and four available positions. Hence there are four different ways we can place these three nodes in le 2. In the same fashion, we have six positions and five nodes to place in level 3. This means we can place f nodes in six different ways. Finally, we have 10 available positions and 4 nodes to place in level 4. That gives (10 4) combinations.
Hence, we have

$$N = \binom{1}{1}\binom{2}{2}\binom{4}{3}\binom{6}{5}\binom{10}{4} = 5040 \text{ different BSTs}$$

However, on Friday, one of your friends brought to my attention, that I failed to specify that the nodes in ea level is supposed to be the same as the BST given in (a).

That changes the result we have computed since there are also BSTs with different numbers of nodes in eac depth level which still provide the same $t_{average}$ and t_{worst}. Hence, the correct result for the question in the ex is as in the following:

BSTs with not only 5 nodes in level #3 and 4 nodes in level 4, but with 3 and 5 nodes in levels 3 and 4 in respective order provide the same $t_{average}$ and t_{worst}. Adding combinations for these kinds of BSTs we get

$$N = \binom{1}{1}\binom{2}{2}\left[\binom{4}{3}\binom{6}{5}\binom{10}{4} + \binom{4}{4}\binom{8}{3}\binom{6}{5}\right] = 5376 \text{ different BSTs}$$

I will accept both answers.

Q4. (20pts)
In terms of *n*, how many times is the printf statement executed in the recursive function afunc given below?

```
.
.
function afunc(n)
if (n>0) {
  afunc(n-1);
  printf("%d\n",n)
  afunc(n-1)
}
return;
.
.
```

Show your calculations!

Answer:

Everytime the function is invoked it executes the printf statement once and invokes itself twice. Hence, the following recurrence is a formulation of the number of times the printf statement is executed in terms of the argument of the function *n*:

$$T(n)=2T(n-1)+1.$$

For $n=0$, the printf statement is not executed. Hence $T(0)=0$.

C.E.: $(x-2)(x-1)=0$
$T(n)=c_1 + c_2 2^n$; $T(0)=0$, $T(1)=1$; $\Rightarrow c_1=-1$, $c_2=1$
Hence ***T(n)= 2^n-1***

Alternative answer :

Values of n	# times printf is executed
1	1
2	3
3	7
4	15
...	...
k	2^k-1

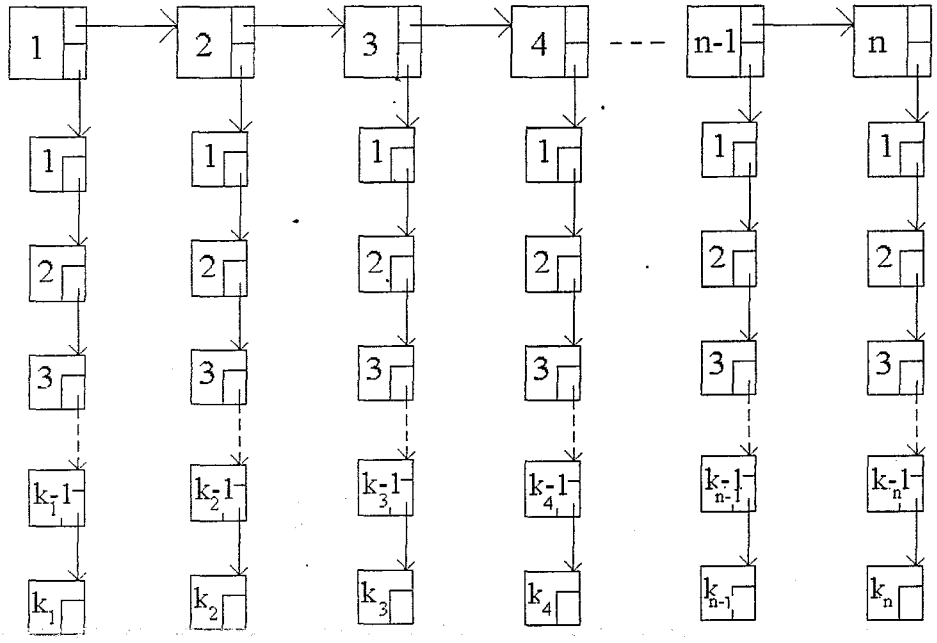
Hence, the printf statement is printed **2^n-1**.

CSE225 DATA STRUCTURES Student Name:_____

Borahan Tümer, Ph.D. **MIDTERM ANSWERS** Student Number:_____

Q5. (30pts)

Consider the data structure below!



There are two types of node structures defined: that of the horizontal linked list and that of the vertical linked list Both types are given in the following:

```
struct vertical_node{
        data_type *data_ptr;
        struct vertical_node *next;
}
struct horizontal_node{
        info_type *info_ptr;
        struct horizontal_node *next;
        struct vertical_node *header_of_vertical_list;
}
```

We assume that *info_type* and *data_type* are previously defined two structures. Each node $i, 1 \le i \le n$, in th horizontal list is a header to a vertical list of length of length $j=k_i$, $1 \le k_i \le m$. Each node in the horizontal list ho class in the computer engineering department. Nodes in each vertical list hold information of students attending class. Note that the total number of the students in the system is m or mathematically expressed

The program that implements and uses this data structure accepts the school id of a student as input and generates a list of classes the student is taking. In other words, for each student presented, the program performs search through the entire structure.

$$\sum_{i=1}^{n} k_i = m.$$

a. (10pts)

Derive the average number of comparisons S per student in terms of n and m.

Hint: Assume that

$$\sum_{i=1}^{n} k_i^2 = 2m$$

$$S=\frac{1}{m}[1+1+2+3+\cdots+k_1+(m-k_1)k_1+1+1+2+3+\cdots+k_2+(m-k_2)k_2+$$
$$+\cdots+1+1+2+3+\cdots+k_n+(m-k_n)k_n]$$

$$S=\frac{1}{m}[\sum_{i=1}^{n}1+\sum_{i=1}^{n}\sum_{j=1}^{k_i}j+\sum_{i=1}^{n}(m-k_i)k_i]=\frac{1}{m}[n+\frac{1}{2}\sum_{i=1}^{n}k_i(k_i+1)+m\sum_{i=1}^{n}k_i-\sum_{i=1}^{n}k_i^{\,2}]$$

$$S=\frac{1}{m}[n-\frac{1}{2}\sum_{i=1}^{n}k_i^{\,2}+(m+\frac{1}{2})\sum_{i=1}^{n}k_i]\cong\frac{1}{m}[n-\frac{1}{2}2m+(m+\frac{1}{2})m]=\frac{n}{m}-\frac{1}{2}+m\in O(m$$

b. (10pts)
Propose an alternative data structure that improves (i.e., shortens) the search time for the task explained above!

Several possible alternative data structures with better performances
1) The students may be held in a BST. The classes that each student takes may be attached to the corresponding tree node as a linked list.

2)The class nodes of the horizontal linked list may be reordered such that the class node with the highest number of students (i.e., with the most crowded vertical linked list attached) becomes the first node, the one with the second highest number of students becomes the second node, etc.

3) In addition to the second improvement, all students in each class may be stored as a BST instead of a vertical linked list.

c. (10pts)
Derive the number of comparisons *S* per student in terms of *n* and *m* for the new data structure proposed in (b).

Solution for the second alternative with the third improvement:
Assumption: all BSTs are complete or almost complete BSTs!

$$S=\frac{1}{m}[\sum_{i=1}^{n}1+\sum_{i=1}^{n}m\log_2 k_i]=\frac{1}{m}[n+m\sum_{i=1}^{n}\log_2 k_i]=\frac{1}{m}[n+m\log_2\prod_{i=1}^{n}k_i]$$